

Local Search

Informed and uninformed searches are mostly technical searches. What matters here is the path we find through the search, the cost of it. So, from which node we visited a path mattered, and we need to store that information. So, we needed backtracking

to go back to the parent node and start exploring rest of the branches.

So, the characteristics of those algorithms are-

- i. Backtracking
- ii. Path mattered
- iii. Perfect goal

In AI, we work with very large graphs and those searches are very expensive.

Especially, memory. As we backtrack, so for large graphs with huge depth, we might face memory overflow issues. We might deploy informed search. However, this is heavily dependent on the nature of the heuristic. These are the issues of informed and uninformed searches.

In AI, there are many problems where we are not concerned about the path, but the solution itself is important. For instance, an AI agent detecting the gender (M/F) from an image, we don't need to worry how it can figure out the gender; as long as we get the correct ans.

wer, we are okay with it.

Sometimes, we also don't care about the perfect solution. For instance, the some AI model might not have 100% accuracy. If it can achieve a threshold level (80%, 85% --), then it's fine.

Here, we don't need to use backtracking.

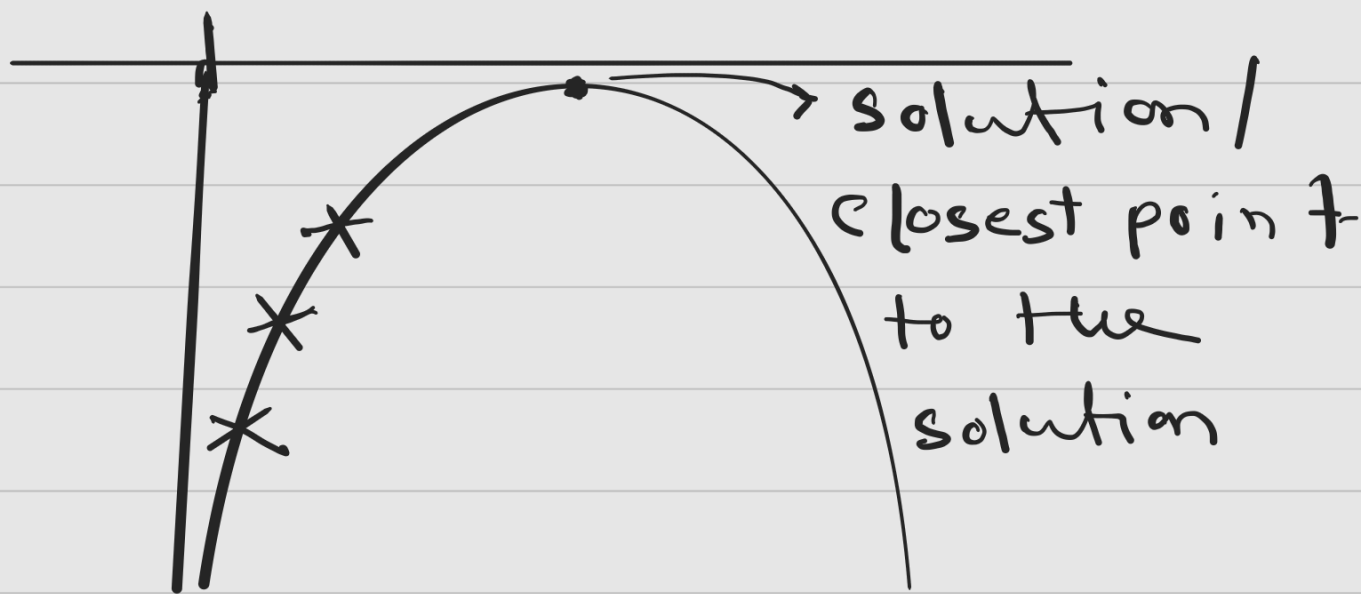
In those scenarios, we

can apply an algorithm which is called the Local Search.

Advantages;

- i. less memory (close to nothing)
- ii. less time (no backtrack)
- iii. Applicable for large graph

One of the algorithms that we study under local search is, Hill Climb search.



← left right →

It's a 2D hill where you can go left or right, and you want find the solution. How will you reach the goal? If at any point going left or right descends

you, that is your goal.

Another example:

x	y
x_1	y_1
x_2	y_2
x_3	y_3
x_4	y_4
x_n	?

Given all the x & y , find

y_n , given x_n .

We can solve it using regression. It is the linear regression.

How to solve it?

Let's plot a graph using the data points.



After plotting it, we try to find a best-fit line.

Using that we answer

any query. We plot x_n

and draw a perpendicular

to the best-fit line and
find the y value.

Any straight line equation -

$$y = m \cdot x + b$$

Diagram illustrating the components of the equation:

- m is labeled "slope" with an arrow pointing to it.
- x is labeled "from table" with an arrow pointing to it.
- b is labeled "y-intercept" with an arrow pointing to it.

Let's assume $b = 0$ (centered line).

$$So, \quad y = \underbrace{m}_{\text{known}} \cdot \underbrace{x}_{\text{known}}$$

Diagram illustrating the components of the equation:

- m is labeled "known" with an arrow pointing to it.
- x is labeled "known" with an arrow pointing to it.
- The result y is labeled "unknown" with an arrow pointing to it.

So, we need to find the correct m -value, and we need to search that.

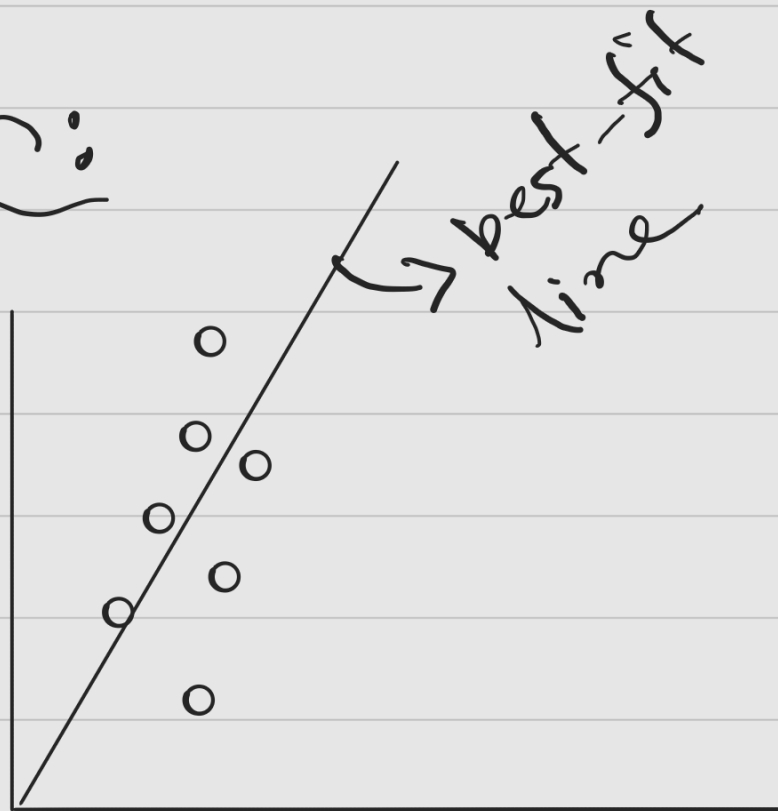
Now, if we think of the search space to find m , it's infinite because m can be $-\infty$ to $+\infty$.

If we set a bound, say 0 to 1 for m , even then there are infinite number of numbers.

Also, it doesn't matter how the m is found, we just need a suitable m from the infinite space.

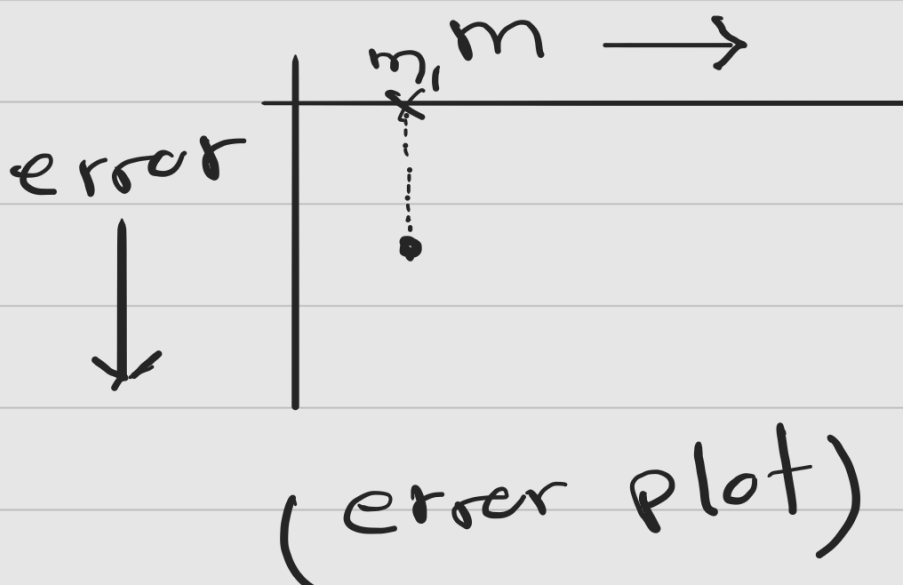
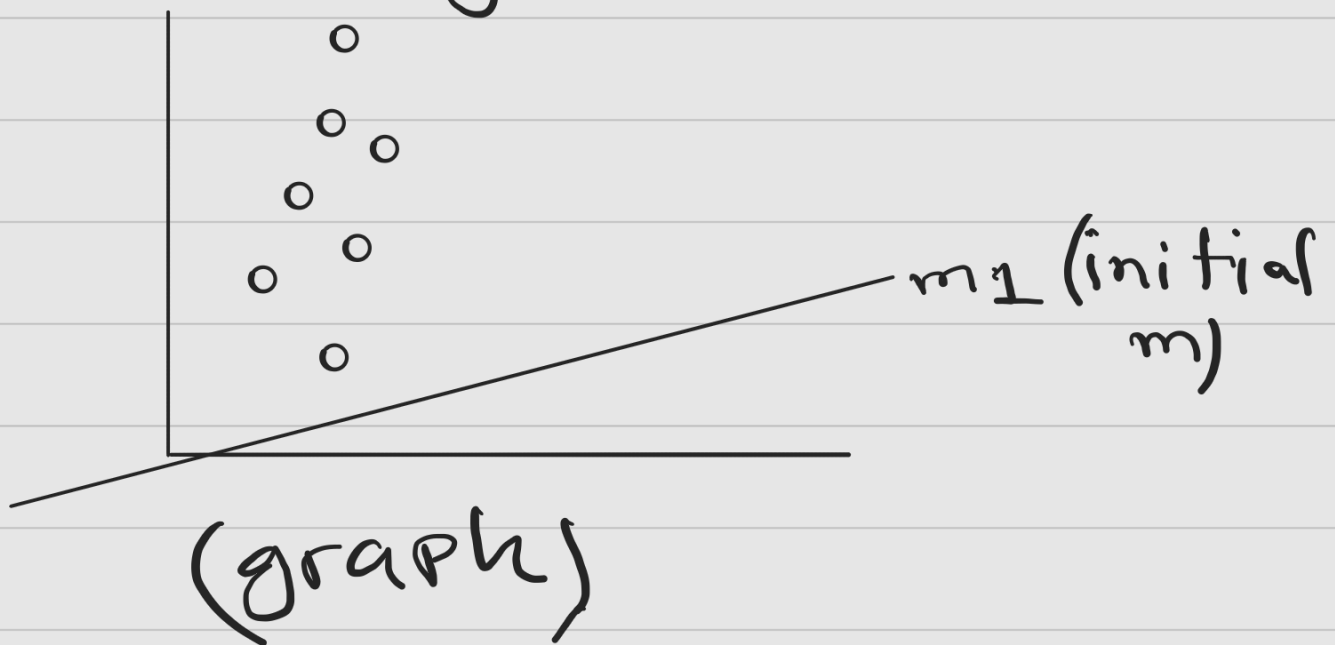
How to solve the hill climb using local search technique?

Solution:



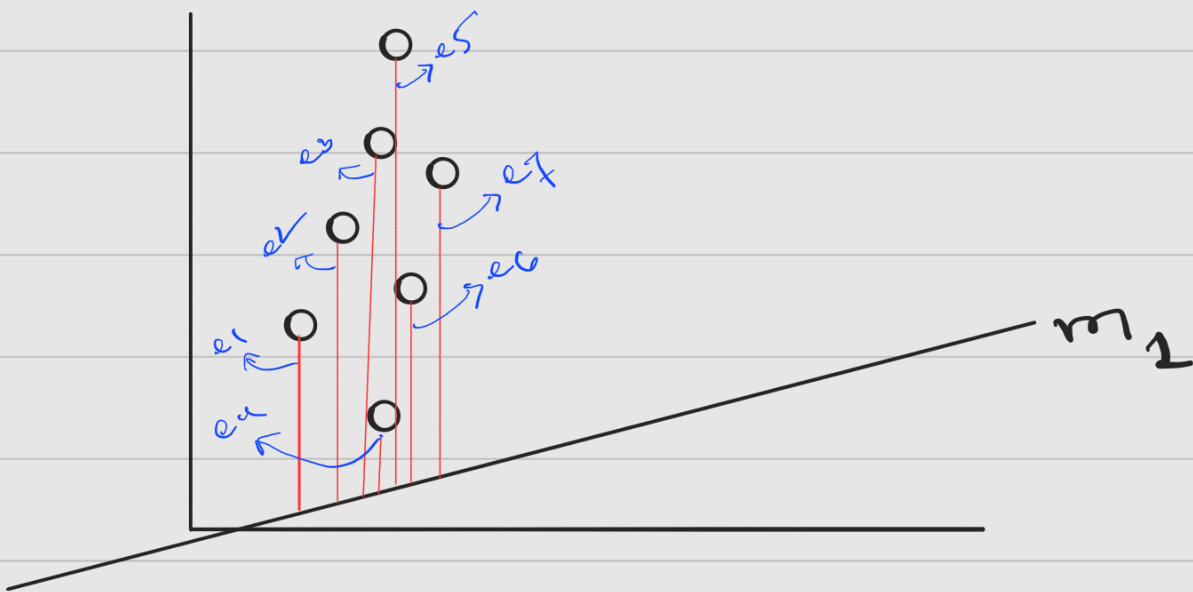
The best fit line has a m ,
and that's the goal of
the problem.

Let's set m equal to
something at random.



How to measure error?

From the points, we measure straight line distance to the best-fit line that's been generated, and add all the error values.

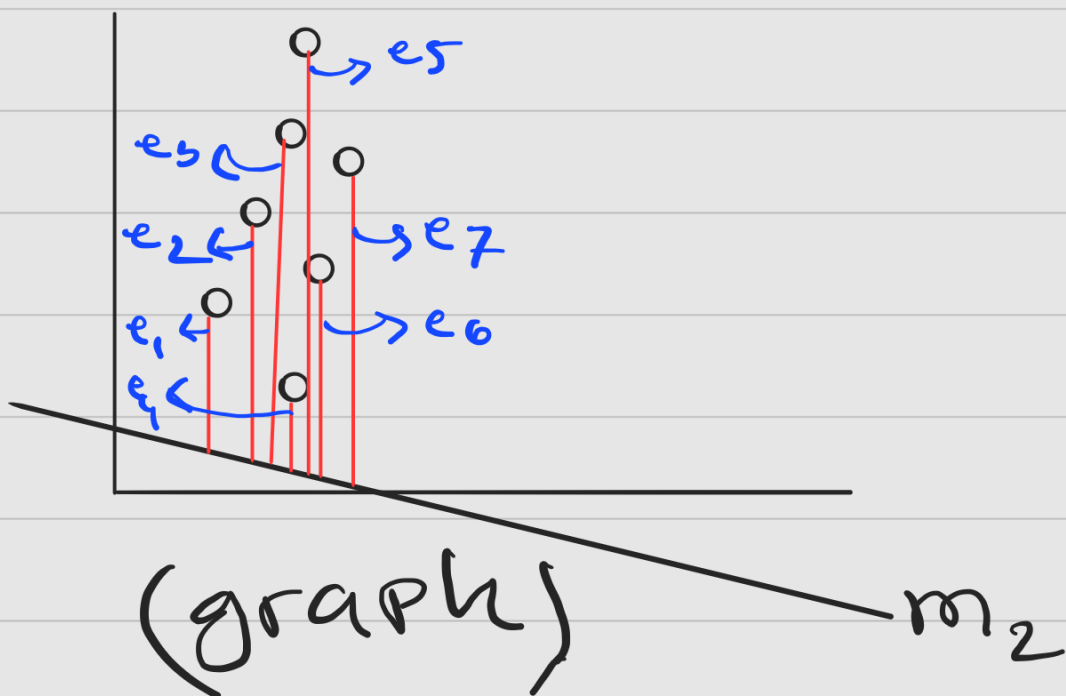


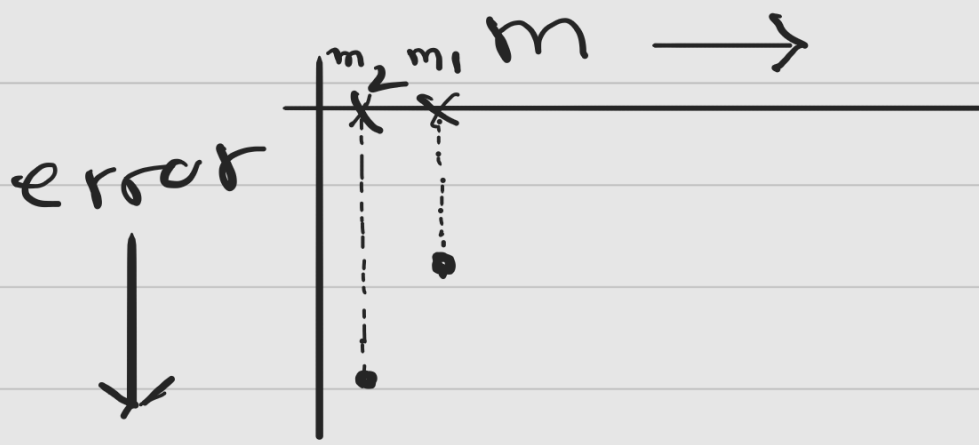
(graph)

$$\text{error} = \sum_{i=1}^n e_i$$

Now, iteratively we can adjust the value of m , for which the errors will be less or minimal.

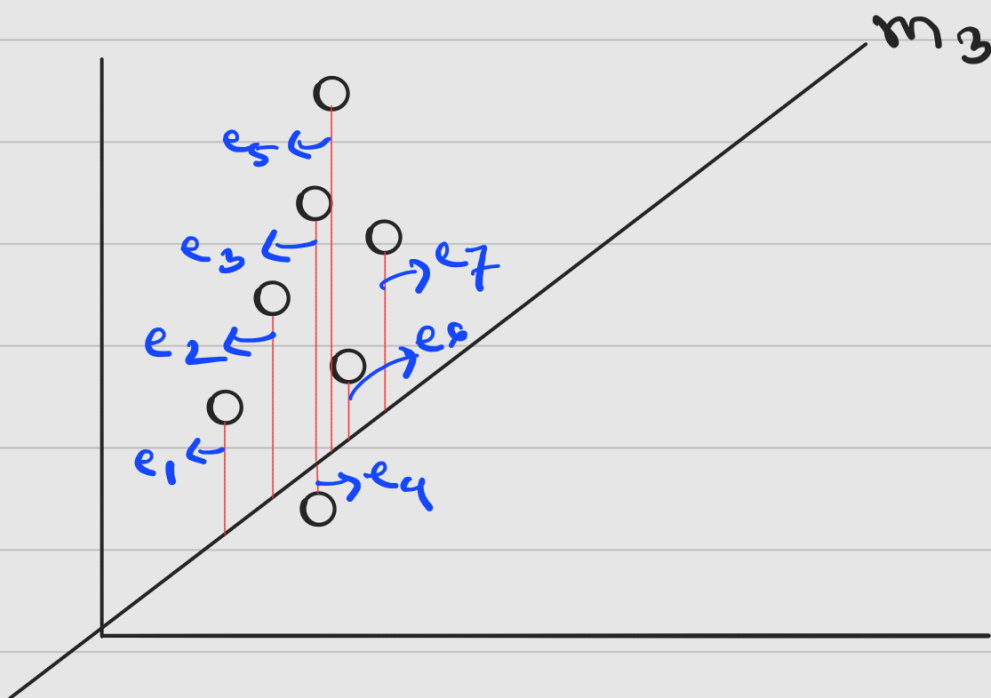
We can increase or decrease m . Let's decrease it.



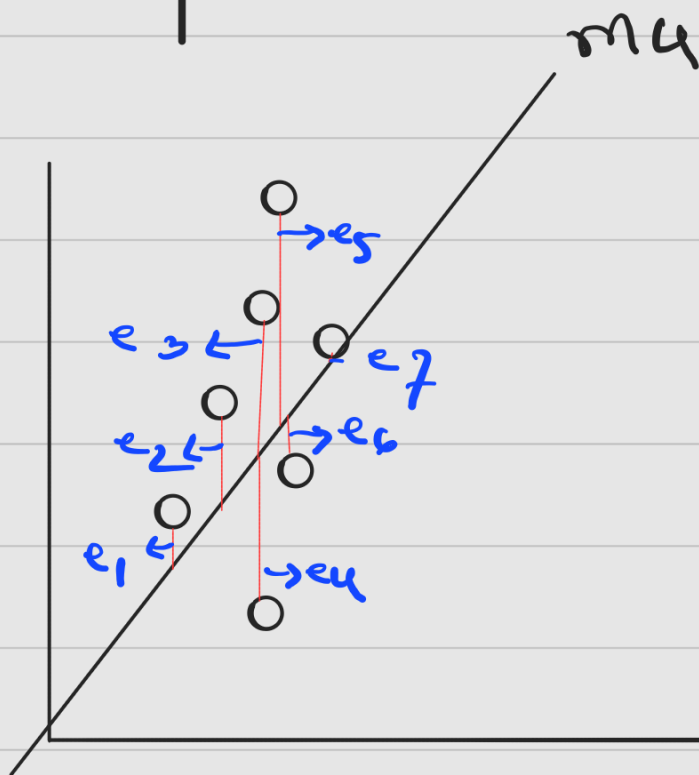
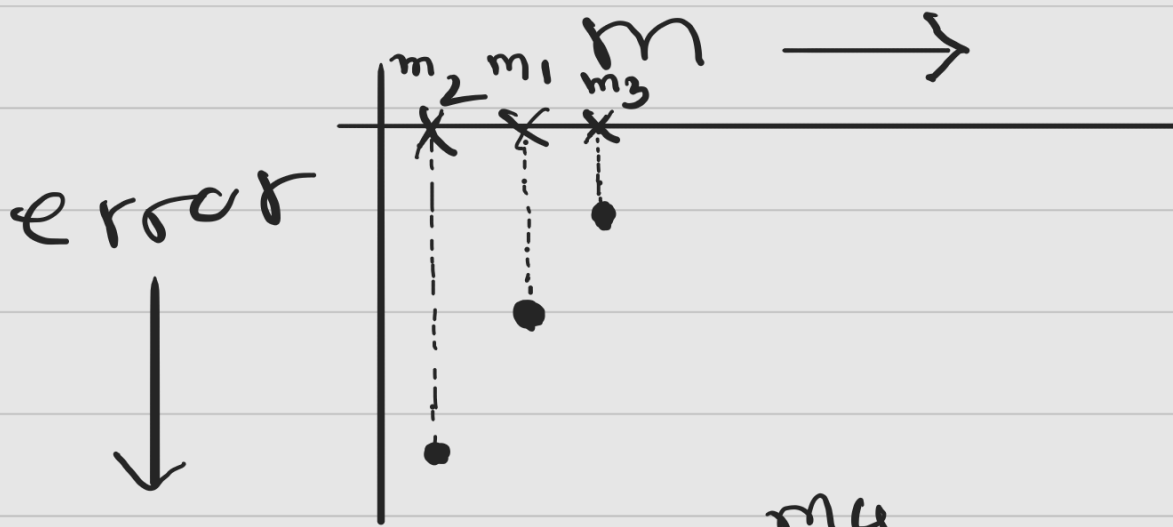


(error plot)

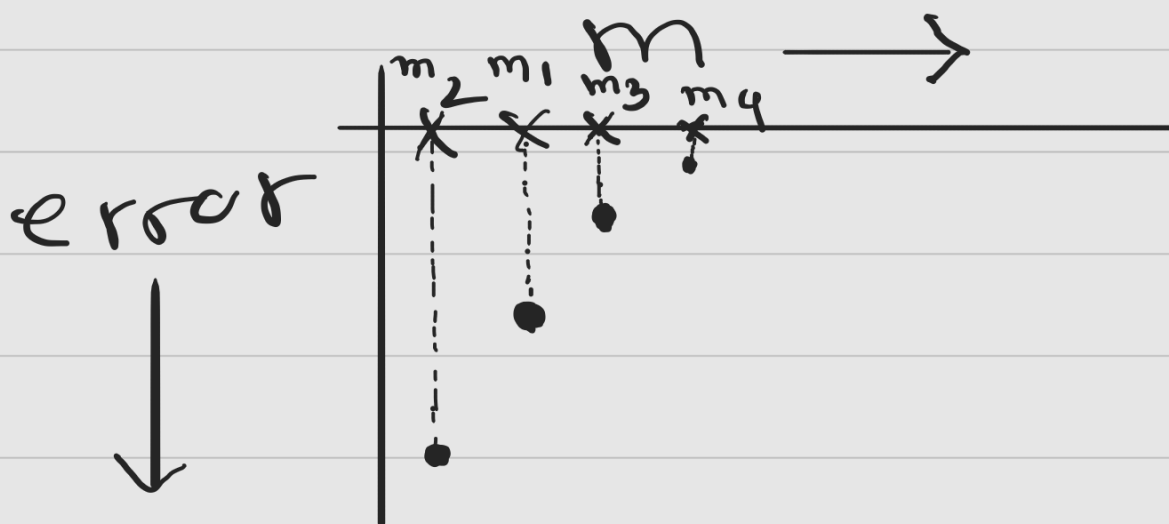
So, we can see the reducing m increase error. So, we should increase m .



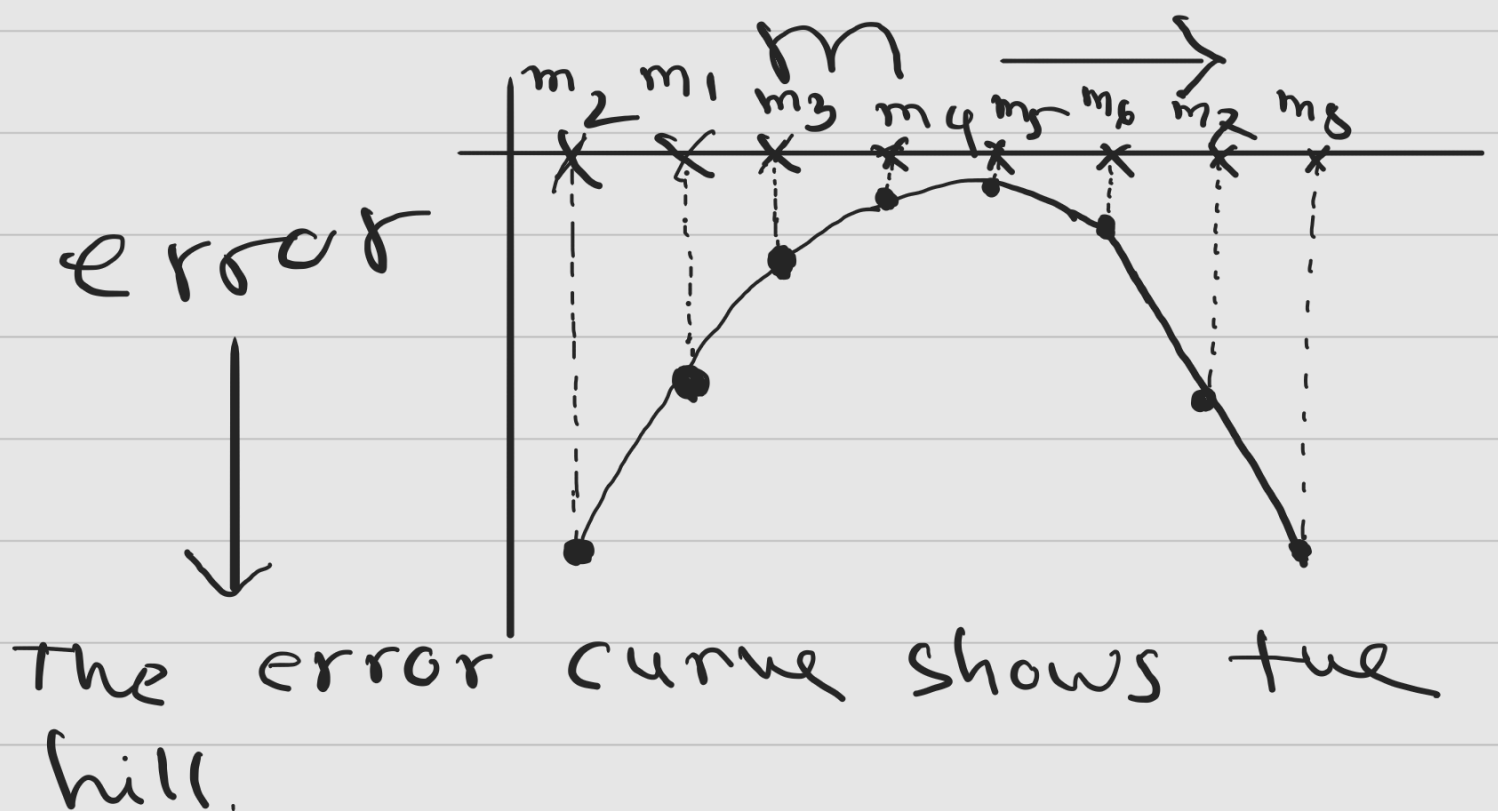
(graph)



(graph)

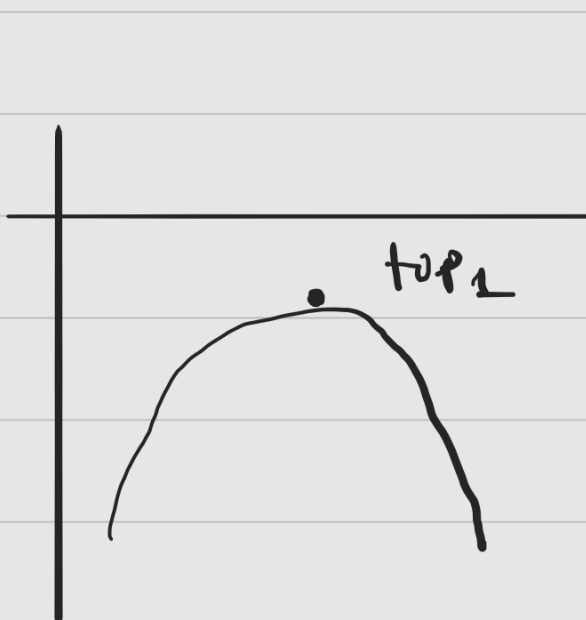


Now, as we see, increasing m is decreasing the error we can keep on increasing m . But, doing so will not reduce the error anymore, rather increase it.

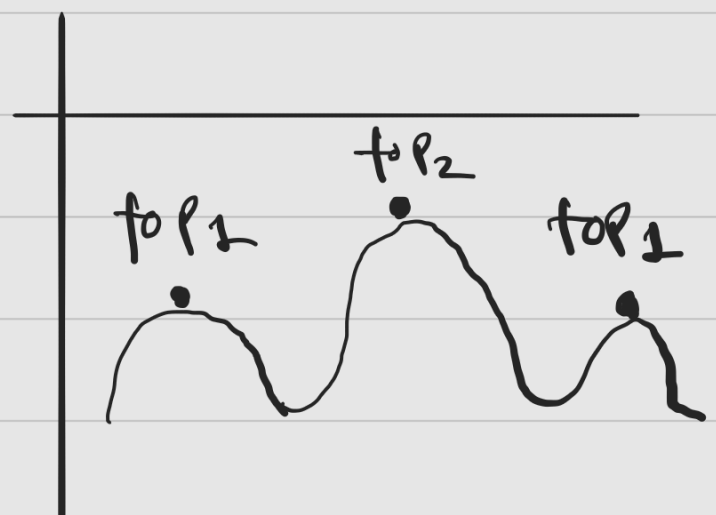


Now, if $b \neq 0$, then we would have solve for two lines. For this one the hill would like a 3D model.

So, we can have single top or multi top in the hill.



single top



multi top

So, if we have n variables, then the shape of the hill will be $n+1$ dimensional.

We talked about the situations where we can apply it, and the advantages.

Now, let's talk about the disadvantages.

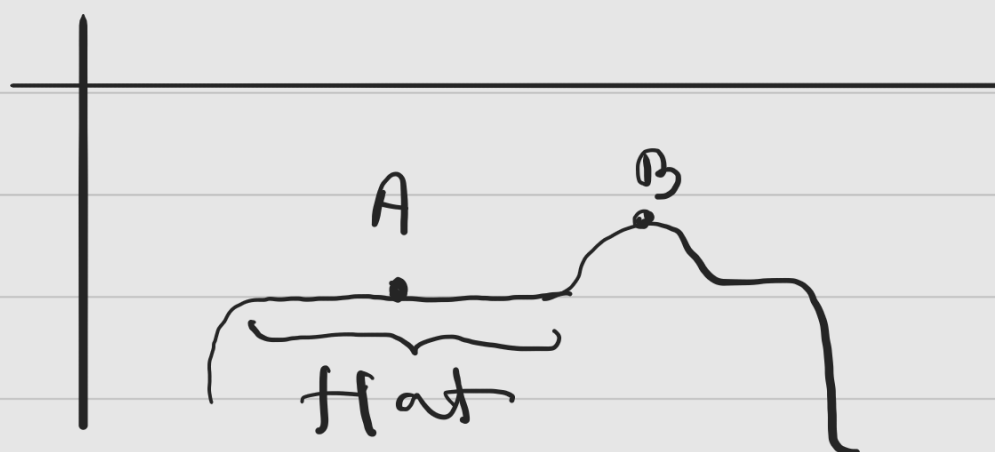
For a single graph multiple top points can be there.



Only one top point that is best is B. That's global maxima and rest are called local maxima. Depending on the start point it might get stuck on local maxima.

First issue - local maxima.

Next,

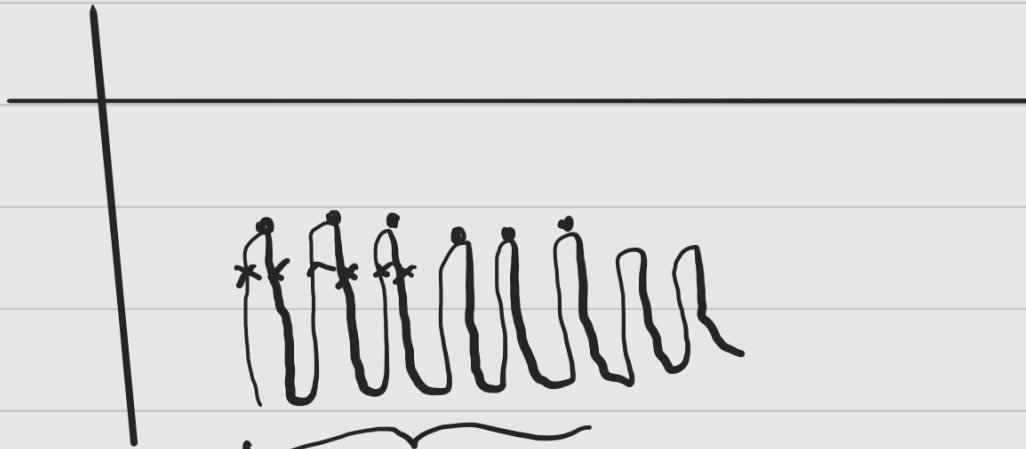


At any point if the

hill climb reaches point
A, it will get stuck
as the neighboring points
will give the same answer.

Second issue - Plateaus

Third issue - Ridges



multiple local maxima

closely packed

Solutions:

i) Random restart

- Probabilistic solution

How many restarts?

If a hill has 5-6 maxima

then we may restart

20-25 times and find

all the maxima.

Next, we can set a thresh-

hold like reaching 80%.

close to the goal (if we

know the goal), then the algorithm will stop.

Next, we may set a maximum iteration (e.g. 10000 times).

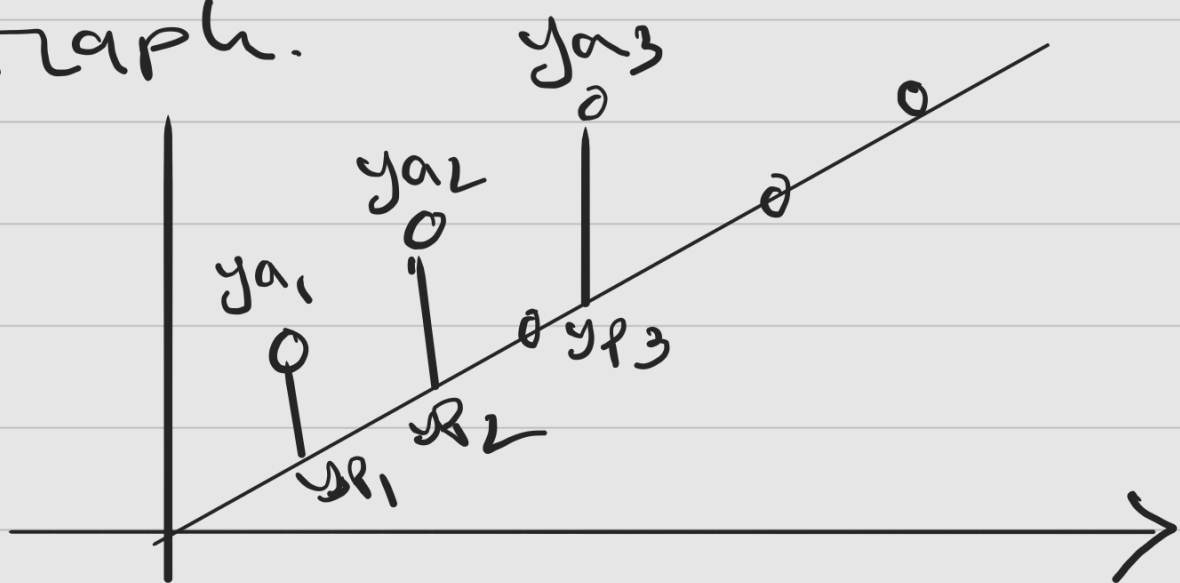
ii) Problem reformulation

Say, the hill is like—



This hill is really complex to navigate through. Here, random restart won't work that good. So, we try to reformulate the problem and change the structure of the problem.

Let's take the regression graph.

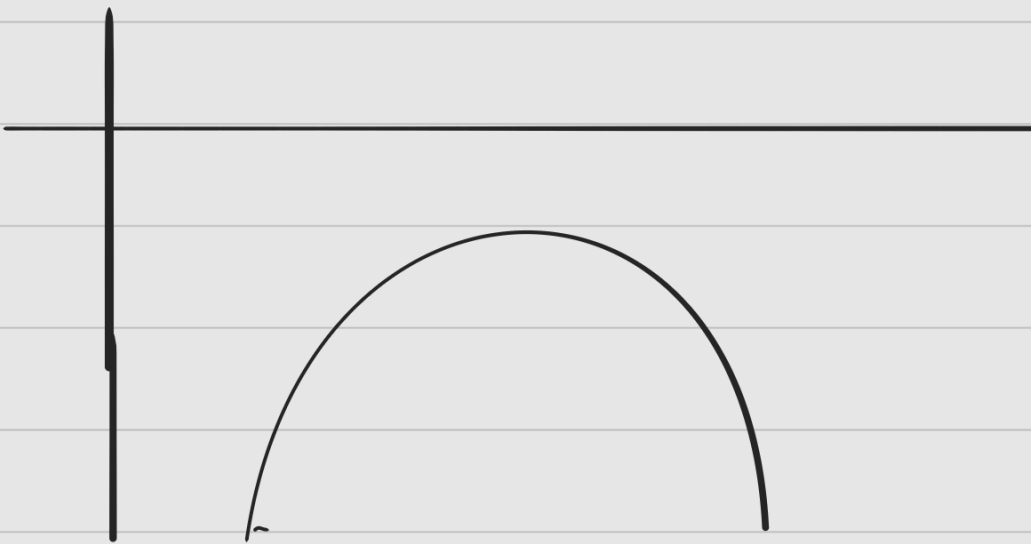


$$\text{Error} = \sum_{i=1}^n (y_a - y_p)^2 \quad \left\{ \begin{array}{l} y = n \\ \text{type} \end{array} \right.$$

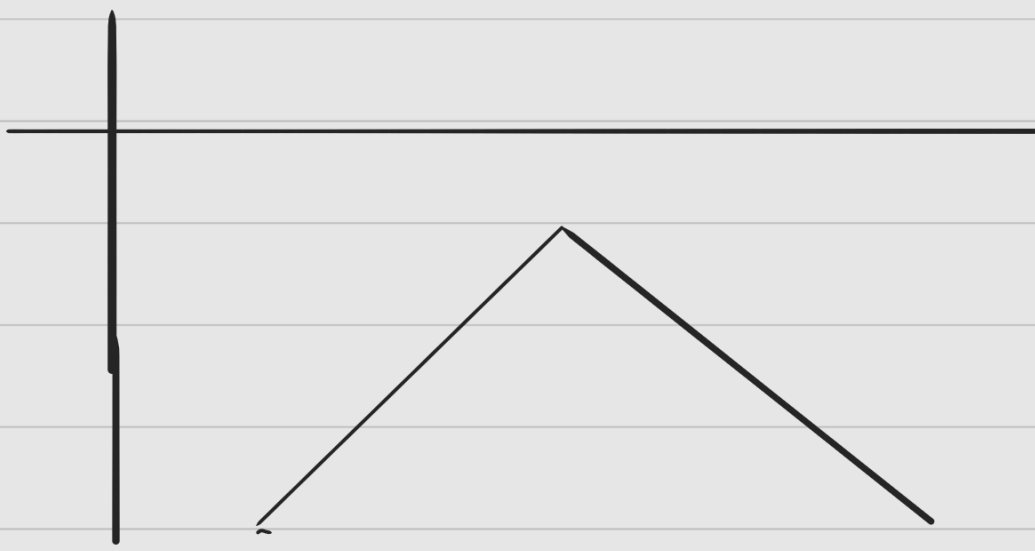
y_a = actual y point

y_p = predicted y point

Using this the error function will be as follow -



Say, now the error function is, $\sum_{i=1}^n |y_a - y_p| \quad \{y = |n|\}$



iii) Simulated Annealing

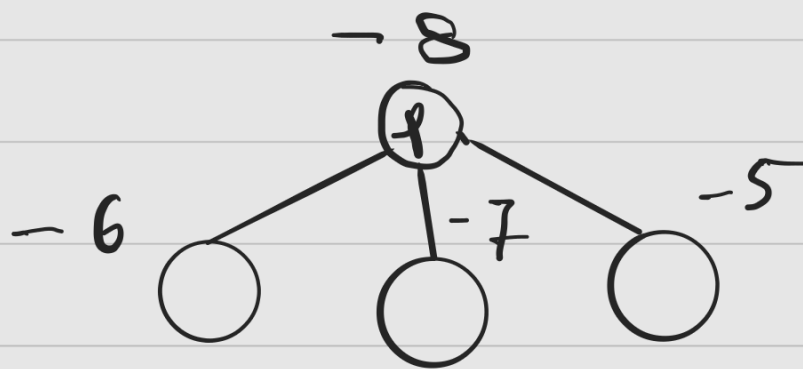
Initially a temperature value will be high and slowly it will go down.

So, when the temperature is high, it will have high probability of accepting a worst state.

And, as it goes down,
the tendency to to
accept worst state will
go down. Finally, after
reaching a threshold
it will stop accepting
worst states.

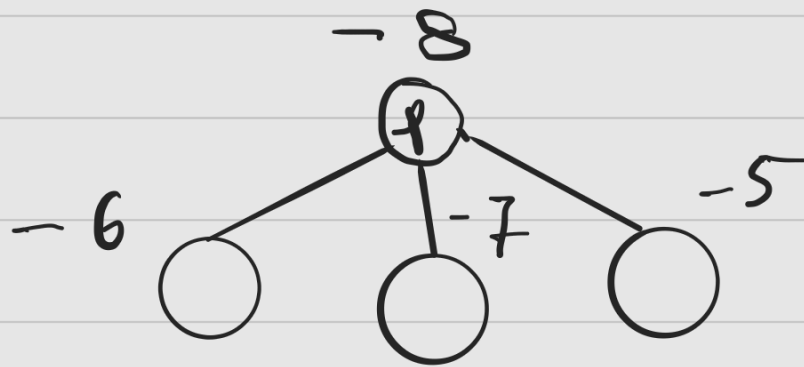
Hill climb type

i) Steepest Hill climb



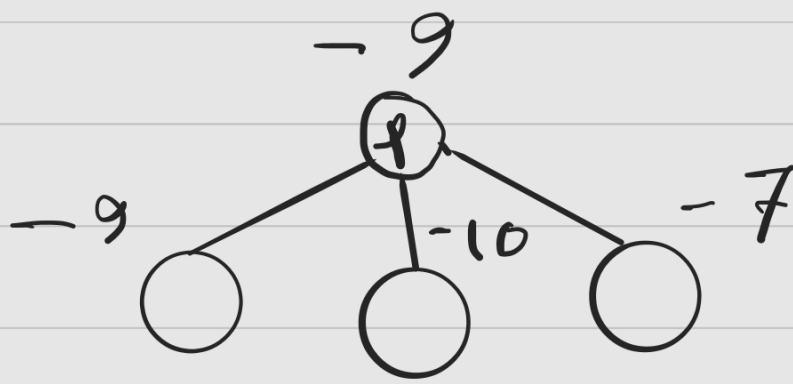
Here, all the child are better than parent and chooses the best one.

ii) Stochastic Hill Climb



It will choose any random one from the better ones.

iii) First Choice Hill Climb



it checks left to right
and chooses the first one
that is better than
the parent.